Лабораторная работа №8 · BFS с MPI + OpenMP

Алгоритмы. Последовательный BFS использует очередь. Граф хранится в CSR: offsets и edges. Ранг 0 читает файл и рассылает CSR всем процессам. На каждом уровне ранг r обрабатывает индексы фронта r, r + P, …; OpenMP распределяет их между потоками. Каждый поток пишет только в свой динамический список, dist лишь читает. После параллельного участка списки объединяются в признаки; Allreduce(MPI_MAX) объединяет признаки всех рангов. Затем каждый ранг одинаково обновляет dist и фронт. Повторные кандидаты удаляются признаками; при пустом фронте обход завершён. MPI вызывается главным потоком, режим MPI_THREAD_FUNNELED.

Данные. Неориентированные графы, s = 0, Python random.Random(2026). S/B/L: кольцо и случайные уникальные рёбра до M = 4N; D: до M = 16N. H: кольцо, связи вершин v ≥ 32 с узлом v mod 32 и случайные рёбра до M = 4N. B приложен как graph.txt. Средняя степень = 2M/N; CSR содержит 2M дуг. Программа также поддерживает ориентированный режим и выбор s.

Среда и замер. Apple M4 (10 ядер: 4P + 6E), 16 ГБ, macOS 26.3.1; Open MPI 5.0.8, Clang 17, libomp 23.1.2, -O2. По одному прогреву и 5 измерениям, среднее время. Чтение, CSR и рассылка вне таймера; MPI_Wtime измеряет BFS с обменами, после Barrier. Для каждого повтора Tpar = max времени рангов через Reduce(MAX); проверка и вывод вне таймера. S = Tseq/Tpar, E = S/(P·T). До 8 рабочих потоков, без привязки к ядрам и превышения 10 ядер.

```
mpicc -O2 -fopenmp bfs.c -o bfs
OMP_NUM_THREADS=4 mpiexec --bind-to none -n 2 ./bfs graph.txt 0 undirected dynamic 5
```

Команда сборки для MPI/GCC; на этом Mac использованы Clang -Xpreprocessor -fopenmp, include/lib libomp и -lomp. Для static заменяется аргумент dynamic. В тестах mpiexec также получал --oversubscribe.

1. Процессы и потоки · граф B, N = 20 000, M = 80 000, dynamic

P	T	P·T	Tseq, с	Tpar, с	S	E
1	2	2	0,0002918	0,0008036	0,363	0,182
2	2	4	0,0002924	0,0009164	0,319	0,080
4	2	8	0,0002904	0,0012920	0,225	0,028
2	1	2	0,0002892	0,0003800	0,761	0,381
2	4	8	0,0002794	0,0011708	0,239	0,030
1	8	8	0,0002896	0,0014174	0,204	0,026
8	1	8	0,0002894	0,0014090	0,205	0,026

При T = 2 сравниваются P = 1, 2, 4; при P = 2 - T = 1, 2, 4. При P·T = 8 сравниваются 1×8, 2×4, 4×2, 8×1. Для каждого запуска приведён собственный последовательный замер того же графа.

2. Размер, структура и планировка · P = 2, T = 4

Граф	N	M	Ст.	Ур.	Tseq*, с	static, с	dynamic, с	S*
S	2000	8000	8	6	0,0000160	0,0004528	0,0003778	0,042
B	20000	80000	8	8	0,0002854	0,0006936	0,0010552	0,270
L	100000	400000	8	9	0,0018708	0,0023722	0,0041486	0,451
D	20000	320000	32	5	0,0005004	0,0007438	0,0010240	0,489
H	20000	80000	8	5	0,0003026	0,0017434	0,0009902	0,306

* Tseq и S относятся к dynamic; сравнение планировок выполнено отдельными сериями. Все вершины этих графов достижимы; число несовпадений расстояний на всех рангах равно 0 во всех повторах. Дополнительно 28 запусков на малых графах совпали с независимым BFS на Python: петли, повторные рёбра, изолированные вершины, направленные рёбра, разные s и обе планировки. Семь ошибочных входов отклонены; AddressSanitizer/UBSan ошибок не выявили.

3. Фронты, балансировка и ограничения

B: размеры фронтов 1, 8, 60, 466, 3245, 11795, 4408, 17; 8 уровней, максимум 11795. При P = 2 ранги обработали 10002/9998 вершин и 80178/79822 дуги. **H:** 1, 630, 3446, 11707, 4216; 5 уровней, максимум 11707; ранги обработали 10001/9999 вершин и 81167/78833 дуги. Статистика всех уровней и рангов выводится программой. Узкий фронт оставляет часть работников без работы; равное число вершин не гарантирует равного числа просмотренных дуг.

На B увеличение P или T не дало ускорения: работа с небольшими списками дешевле синхронизации, объединения кандидатов и сканирования N признаков на каждом уровне. Даже при 8 потоках сочетания P/T отличаются стоимостью MPI и OpenMP. На H dynamic оказался быстрее static, на B/L/D - медленнее; гибкое распределение может помочь при вершинах высокой степени, но имеет накладные расходы. Это результаты коротких локальных замеров, чувствительных к планированию ОС; рост степени меняет и стоимость обработки, и число уровней.

На каждом из K уровней Allreduce обрабатывает N целых чисел: логический объём входов на ранг 4NK байт (для B: 640 000 байт), а фактический трафик зависит от алгоритма MPI. Плотный обмен избыточен при малом числе кандидатов и особенно заметен при больших N и узких фронтах. Копия CSR занимает примерно 4(N + 1 + 2M) байт на ранг, дополнительно нужны массивы BFS и списки; суммарная память растёт с P. Альтернатива - удалять повторы локально и обмениваться только списками через Allgatherv, учитывая их размеры и стоимость объединения.